

Contents

QSPI/SPI PSRAM Controller — Documentation Index	3
Document Structure	3
Quick Reference	3
Module Hierarchy	3
Clock Domains at a Glance	4
Register Map Summary	4
01 — Overview	4
1.1 Introduction	4
1.2 Key Features	4
1.3 Top-Level Parameters	6
1.4 Sub-System Summary	6
Module Hierarchy	7
1.5 AXI Controller Detail	7
1.6 CSR Block Detail	8
1.7 QSPI FSM Detail	8
02 — Port Description	8
2.1 Clock and Reset	8
2.2 AXI4 Slave Interface	9
Write Address Channel (AW)	9
Write Data Channel (W)	10
Write Response Channel (B)	10
Read Address Channel (AR)	10
Read Data Channel (R)	11
2.3 APB Slave Interface	11
2.4 QSPI Pad Interface	12
2.5 Port Connectivity Notes	12
03 — Clocks and Resets	13
3.1 Clock Domain Overview	13
3.2 SCLK and QSPI Output Clock	14
3.3 Reset Strategy	14
3.4 Clock Frequency Constraints	15
3.5 CDC Summary	15
04 — Clock Domain Crossing (CDC)	15
4.1 ACLK ↔ SCLK: Asynchronous FIFO (Toggle-Vector Scheme)	15
4.1.1 FIFO Instances	15
4.1.2 Toggle-Vector Occupancy Scheme	16
4.1.3 Required STA Constraints	17
4.2 PCLK ↔ SCLK: 4-Phase Handshake (CSR)	17
4.2.1 Mechanism Overview	17
4.2.2 Signal Flow	17
4.2.3 Key Signals in m_vlsi_qspi_csr	18
4.2.4 Latency	19
4.2.5 Required STA Constraints	19

4.3 CDC Waiver Guidance	19
05 — Configuration & Status Registers (CSR)	19
Register Map	20
0x00 — ctrl (Control)	21
0x04 — wd (Width Descriptor)	21
0x08 — read (Read Command)	22
0x0C — write (Write Command)	23
0x10 — wr_dummy (Write Dummy Count)	24
0x14 — rd_dummy (Read Dummy Count)	24
0x18 — mode_status (Mode Status)	25
0x1C — independent (Independent Command)	25
Register Access Sequence (req/ack Pattern)	26
06 — Functional Description	27
6.1 QSPI FSM	27
6.1.1 State Summary	27
6.1.2 State Transitions	27
6.1.3 Phase Details	29
6.2 Arbitration	29
6.2.1 Arbitration Logic	29
6.2.2 Request Conditions	29
6.3 Write Flow	30
6.4 Read Flow	30
6.5 Independent Command Flow	31
6.6 XIP Mode	31
6.7 auto_data_wd Mode	31
6.8 Write Response Error (SLVERR)	31
07 — Software Integration Guide	32
7.1 Initialization Sequence	32
7.1.1 Register Configuration Order	32
7.2 Normal AXI Read/Write	32
7.3 Changing Command Opcodes at Run Time	33
7.4 Independent (Command-Only) Transaction	33
7.5 Switching from SPI to QSPI Mode	33
7.6 Switching from QSPI to SPI Mode	34
7.7 APB Access Latency Notes	34
7.8 Polling vs. Interrupt-Driven Operation	35
7.9 Bare-Metal Driver Pseudocode	35
08 — STA Constraints Reference	36
8.1 ACLK ↔ SCLK (Async FIFO Toggle-Vector Synchronizers)	36
8.2 PCLK ↔ SCLK (CSR 4-Phase Handshake Synchronizers)	36
8.3 False Path Declarations	37
8.4 Reset Path Constraints	37
8.5 Synchronizer Cell Attributes	37
8.6 Constraint Summary Table	38

QSPI/SPI PSRAM Controller — Documentation Index

IP Name: m_vlsi_qspi_top
Revision: 2.0
Date: 2026-06
Authors: Ethan, Link
Organization: VLSI Technology

Document Structure

File	Contents
01_overview.md	IP overview, key features, top-level parameters, block diagram
02_ports.md	Complete port list — AXI4, APB, QSPI, clock/reset
03_clock_reset.md	Clock domains, reset strategy
04_cdc.md	Clock Domain Crossing mechanisms and timing constraints
05_csr.md	Configuration & Status Registers — full bit-field reference
06_functional.md	FSM states, arbitration, write/read/independent command flows
07_software.md	Software integration guide — initialization, runtime sequences
08_constraints.md	STA constraints reference

Quick Reference

Module Hierarchy

```

m_vlsi_qspi_top
├── m_vlsi_axi4_sram          (AXI Controller)
│   ├── m_vlsi_axfsm ×2      (AW and AR channel FSMs)
│   ├── m_vlsi_axi_bus_logic (W / R / B channel logic, ACLK side)
│   ├── m_vlsi_axi_sclk_logic (memory-side logic, SCLK side)
│   ├── m_vlsi_arbiter        (round-robin read/write arbiter)
│   └── m_vlsi_async_fifo ×5  (AWFIFO, WFIFO, ARFIFO, RFIFO, BFIFO)
├── m_vlsi_qspi_csr          (CSR – APB slave, 4-phase handshake CDC)
│   └── m_vlsi_synch ×4       (2-stage synchronizers)

```

└─ m_vlsi_qspi_fsm (QSPI transaction engine)

Clock Domains at a Glance

Domain	Clock	Reset	Key logic
ACLK	i_axi_clk	i_axi_rstn	AXI slave interface, async FIFO AXI side
PCLK	i_apb_clk	i_apb_rstn	APB CSR interface
SCLK	i_qspi_clk	i_qspi_rstn	QSPI FSM, CSR register file, async FIFO SCLK side

Register Map Summary

Offset	Name	Description
0x00	ctrl	Enable, auto data-width, 2-byte command mode
0x04	wd	Width descriptor (address/data width)
0x08	read	Read command opcode
0x0C	write	Write command opcode
0x10	wr_dummy	Write dummy cycle count
0x14	rd_dummy	Read dummy cycle count
0x18	mode_status	Current interface mode (read-only)
0x1C	independent	Independent (command-only) transaction

01 — Overview

1.1 Introduction

The QSPI/SPI PSRAM Controller (m_vlsi_qspi_top) is a hardware IP developed by VLSI Technology that bridges an AXI4 memory-mapped bus to an external PSRAM device over a QSPI (Quad SPI) or standard SPI interface.

The IP accepts AXI4 read and write transactions from a host processor, translates them into the command–address–dummy–data phase sequence required by QSPI/SPI PSRAMs, and drives the I/O pads accordingly. A separate APB slave interface provides run-time configuration of command opcodes, data widths, dummy-cycle counts, and operating mode through a set of Control and Status Registers (CSRs).

1.2 Key Features

- AXI4 full slave interface — AW, W, B, AR, R channels with burst support (FIXED, INCR, WRAP)
- APB slave interface — CSR configuration and status readback
- QSPI (4-bit I/O) and SPI (1-bit I/O) protocol support

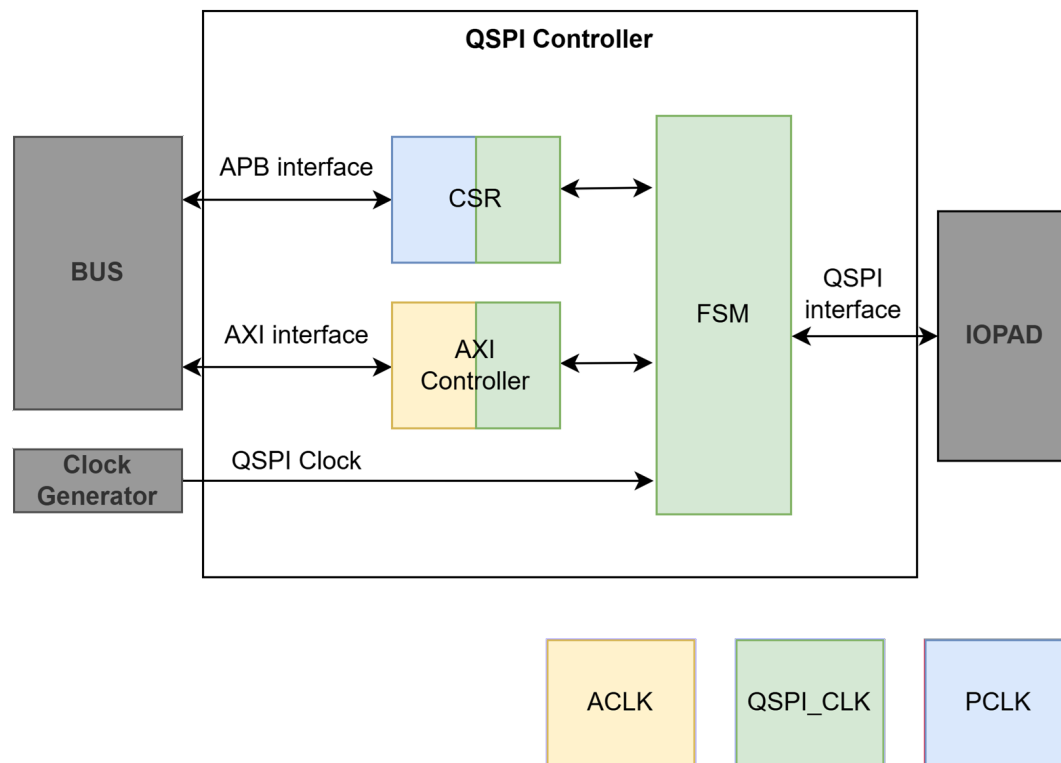


Figure 1: Top-level block diagram

- Software-controlled independent commands — e.g., SPI→QSPI mode switching
- Dynamic reconfiguration — read/write command opcodes, address/data widths, and dummy cycles can be changed at run time without re-synthesis
- XIP (Execute-In-Place) — inferred automatically from AXI burst semantics; no dedicated control bit required
- Round-robin arbitration — concurrent read and write requests are served fairly in the SCLK domain
- Three fully independent clock domains — ACLK, PCLK, and SCLK; no assumed phase or frequency relationship between any pair

1.3 Top-Level Parameters

Parameter	Default	Description
PARAM_AXI_DATA_WD	32	AXI data bus width (bits)
PARAM_AXI_ADDR_WD	24	AXI address bus width (bits)
PARAM_AXI_ID_WD	4	AXI transaction ID width (bits)
PARAM_AXI_LEN_WD	8	AXI burst length field width (bits)
PARAM_AXI_FIFO_DEPTH	8	Depth of each asynchronous FIFO

1.4 Sub-System Summary

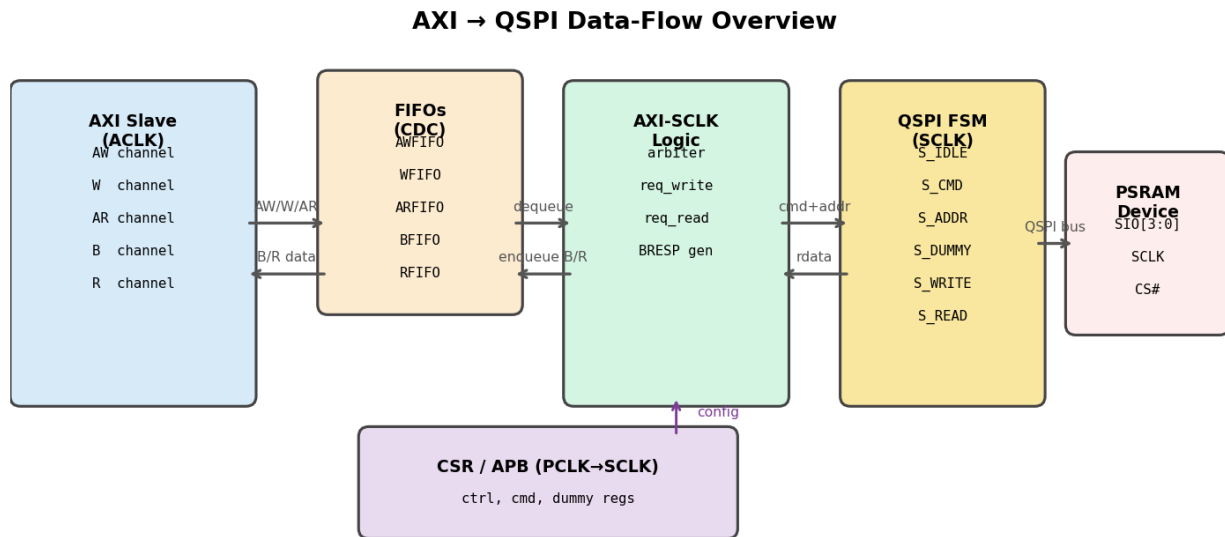


Figure 2: AXI dataflow diagram

Sub-system	Module(s)	Clock domain	Role
AXI Controller	m_vlsi_axfsm ×2, m_vlsi_axi_bus_logic, m_vlsi_axi_sclk_logic, m_vlsi_arbiter, m_vlsi_async_fifo ×5	ACLK / SCLK	Accepts AXI4 transactions, crosses to SCLK, drives PSRAM interface
CSR	m_vlsi_qspi_csr, m_vlsi_synch ×4	PCLK / SCLK	APB register file with 4-phase handshake CDC
QSPI FSM	m_vlsi_qspi_fsm	SCLK	Core transaction engine: CMD-ADDR-DUMMY-DATA sequencing
Arbiter	m_vlsi_arbiter	SCLK	Round-robin read/write arbitration
Synchronizers	m_vlsi_synch, m_vlsi_multi_synch	Various	2-stage flip-flop synchronizers for FIFO CDC and CSR CDC

Module Hierarchy

```

m_vlsi_qspi_top
├── m_vlsi_axi4_sram                (AXI Controller)
│   ├── m_vlsi_axfsm ×2            (AW and AR channel FSMs)
│   ├── m_vlsi_axi_bus_logic       (W / R / B channel logic, ACLK side)
│   ├── m_vlsi_axi_sclk_logic      (memory-side logic, SCLK side)
│   ├── m_vlsi_arbiter             (round-robin arbiter)
│   └── m_vlsi_async_fifo ×5       (AWFIFO, WFIFO, ARFIFO, RFIFO, BFIFO)
├── m_vlsi_qspi_csr                (CSR block)
│   └── m_vlsi_synch ×4            (2-stage synchronizers for 4-
phase handshake)
└── m_vlsi_qspi_fsm                (QSPI transaction engine)

```

1.5 AXI Controller Detail

The AXI Controller (m_vlsi_axi4_sram) is the AXI4 slave front-end. It accepts read and write transactions from the AXI master on the i_axi_clk (ACLK) domain and transfers requests to the QSPI FSM on the i_qspi_clk (SCLK) domain through five asynchronous FIFOs.

Sub-modules:

- `m_vlsi_ax fsm (×2)` — One instance per address channel (AW and AR). Each handles AXI handshaking, per-beat burst address generation, and FIFO push control.
 - `m_vlsi_axi_bus_logic` — Manages AXI data channel handshaking: receives write data (W channel), drives read data (R channel), and drives write response (B channel). Operates on the ACLK side of the async FIFOs.
 - `m_vlsi_axi_sclk_logic` — Runs in the SCLK domain. Pops requests from async FIFOs and drives the memory-side interface (address, write data, read data, control signals) toward the QSPI FSM.
 - `m_vlsi_arbiter` — Round-robin arbitration between concurrent read and write requests in the SCLK domain.
 - `m_vlsi_async_fifo (×5)` — Asynchronous FIFOs bridging ACLK and SCLK: AWFIFO, WFIFO, ARFIFO, RFIFO, BFIFO.
-

1.6 CSR Block Detail

The CSR block (`m_vlsi_qspi_csr`) provides software access to all IP configuration and status registers through the APB slave interface (PCLK domain). Register values are consumed by the QSPI FSM in the SCLK domain.

Because PCLK and SCLK are asynchronous, every APB access uses a 4-phase handshake synchronization. This keeps PREADY deasserted until the register update has safely crossed into the SCLK domain. Four `m_vlsi_synch` 2-stage synchronizer instances implement the handshake signals.

1.7 QSPI FSM Detail

The QSPI FSM (`m_vlsi_qspi_fsm`) is the core transaction engine. It runs entirely in the SCLK domain and is responsible for:

- Receiving AXI read/write requests from the SCLK-side AXI logic
- Accepting CSR configuration: command opcodes, dummy cycle counts, SPI/QSPI mode, enable
- Generating QSPI transactions: CS assert/deassert, command phase, address phase, dummy phase, data phase
- Driving pad-side signals: `o_qspi_csn`, `io_qspi_data[3:0]` (output enable + data)
- Sampling incoming data from `io_qspi_data[3:0]` and returning it to the AXI Controller
- Handling independent command mode
- Reporting current PSRAM mode status back to the CSR (`mode_status.current`)

02 — Port Description

2.1 Clock and Reset

Port	Dir	Width	Description
i_axi_clk	In	1	AXI bus clock (ACLK domain)
i_axi_rstn	In	1	AXI active-low synchronous reset
i_apb_clk	In	1	APB bus clock (PCLK domain)
i_apb_rstn	In	1	APB active-low synchronous reset
i_qspi_clk	In	1	QSPI/internal logic clock (SCLK domain). Also drives o_qspi_sclk directly.
i_qspi_rstn	In	1	QSPI active-low synchronous reset

All resets are active-low and synchronous within their respective clock domain. No cross-domain reset synchronization is provided internally — the user is responsible for correct reset sequencing.

2.2 AXI4 Slave Interface

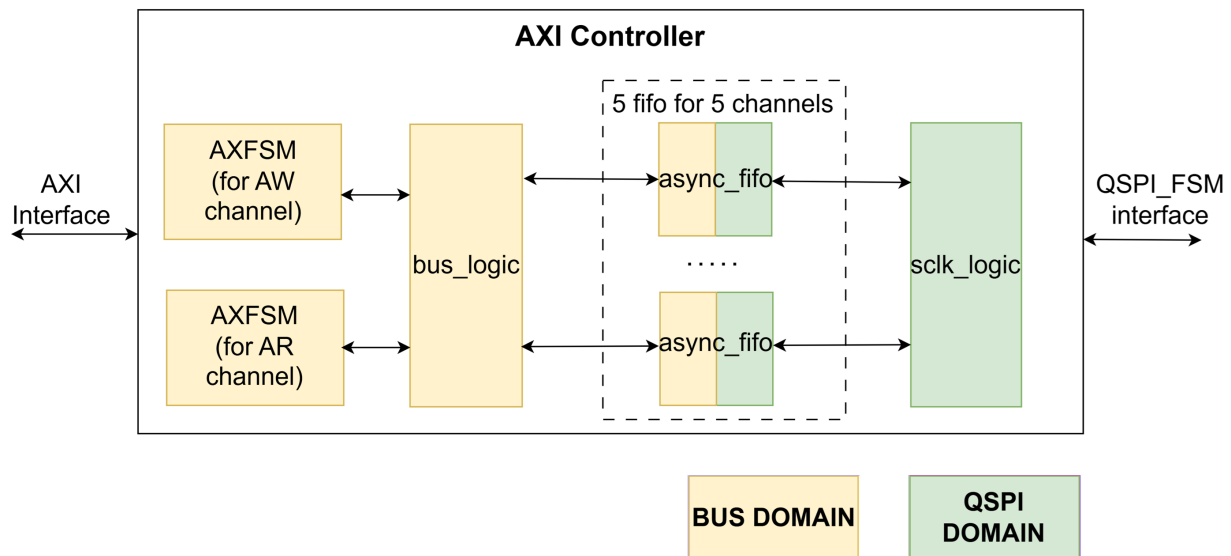


Figure 3: AXI interface diagram

Write Address Channel (AW)

Port	Dir	Width	Description
i_awvalid	In	1	Write address valid
o_awready	Out	1	Write address ready
i_awaddr	In	PARAM_AXI_ADDR_WD	Write address
i_awid	In	PARAM_AXI_ID_WD	Write transaction ID
i_awlen	In	PARAM_AXI_LEN_WD	Burst length minus 1 (number of beats = awlen + 1)
i_awsz	In	3	Burst size: number of bytes per beat encoded as log2
i_awburst	In	2	Burst type: 2'b00=FIXED, 2'b01=INCR, 2'b10=WRAP

Write Data Channel (W)

Port	Dir	Width	Description
i_wvalid	In	1	Write data valid
o_wready	Out	1	Write data ready
i_wdata	In	PARAM_AXI_DATA_WD	Write data
i_wstrb	In	PARAM_AXI_DATA_WD/8	Write byte strobes (1 bit per byte lane)
i_wlast	In	1	Last beat of write burst

Write Response Channel (B)

Port	Dir	Width	Description
o_bvalid	Out	1	Write response valid
i_bready	In	1	Write response ready
o_bid	Out	PARAM_AXI_ID_WD	Write response ID (echoes awid)
o_bresp	Out	2	Write response code: 2'b00=OKAY, 2'b10=SLVERR (burst-length mismatch)

Read Address Channel (AR)

Port	Dir	Width	Description
i_arvalid	In	1	Read address valid
o_arready	Out	1	Read address ready
i_araddr	In	PARA_AXI_ADDR_WD	Read address
i_arid	In	PARA_AXI_ID_WD	Read transaction ID
i_arlen	In	PARA_AXI_LEN_WD	Burst length minus 1
i_arsize	In	3	Burst size
i_arburst	In	2	Burst type: 2'b00=FIXED, 2'b01=INCR, 2'b10=WRAP

Read Data Channel (R)

Port	Dir	Width	Description
o_rvalid	Out	1	Read data valid
i_rready	In	1	Read data ready
o_rdata	Out	PARA_AXI_DATA_WD	Read data
o_rid	Out	PARA_AXI_ID_WD	Read response ID (echoes arid)
o_rlast	Out	1	Last beat of read burst
o_rresp	Out	2	Read response code: 2'b00=OKAY

2.3 APB Slave Interface

Port	Dir	Width	Description
i_psel	In	1	APB select — selects this slave
i_penable	In	1	APB enable — second cycle of APB transfer
i_pwrite	In	1	Transfer direction: 1=write, 0=read
i_paddr	In	16	APB address (byte-addressed; only [4:0] used for register offset)
i_pwdata	In	32	APB write data
i_pstrb	In	4	APB write strobes (byte-enable, one bit per byte)
i_pprot	In	3	APB protection attributes

Port	Dir	Width	Description
i_protect_en	In	1	Enable protection check on pprot[1] (privileged access only)
i_slverr_en	In	1	Enable SLVERR response on address or strobe violations
o_prdata	Out	32	APB read data
o_pready	Out	1	APB ready — held low until CDC round-trip completes
o_pslverr	Out	1	APB slave error

o_pready is deasserted for the duration of the 4-phase CDC handshake. Minimum stall is approximately 2 SCLK cycles + 2 PCLK cycles. See [04_cdc.md](#) for details.

2.4 QSPI Pad Interface

Port	Dir	Width	Description
o_qspi_sclk	Out	1	QSPI clock output — direct wire: assign o_qspi_sclk = i_qspi_clk
o_qspi_csn	Out	1	QSPI chip-select (active low)
io_qspi_data	Inout	4	Quad SPI data lines DQ[3:0]; tri-stated during dummy and idle phases

o_qspi_sclk is a combinational pass-through — there is no clock gating. The SCLK output runs continuously whenever i_qspi_clk is toggling, regardless of whether a transaction is in progress.

2.5 Port Connectivity Notes

- io_qspi_data[0] is MOSI in SPI mode; all four bits are driven in QSPI mode.
- io_qspi_data[1] is MISO in SPI mode; it is driven by the IP during QSPI writes.
- The output-enable signals for io_qspi_data are controlled internally by the FSM based on the current phase (command/address/write = driven; dummy/read = high-Z).
- The IP does not include I/O pad cells. The user must instantiate appropriate pad cells for io_qspi_data with separate output-enable pins in the SoC integration wrapper.

03 — Clocks and Resets

3.1 Clock Domain Overview

The IP operates across three fully independent, asynchronous clock domains. No phase or frequency relationship is assumed between any pair of clocks.

Clock Domain Overview

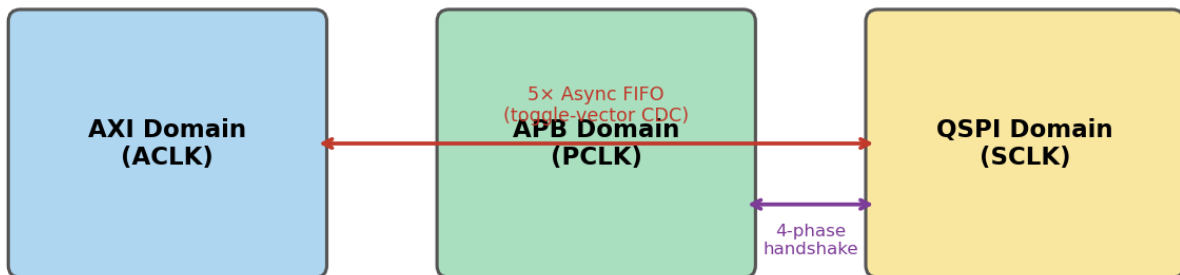


Figure 4: Clock domains diagram

Domain	Clock signal	Reset signal	Description
ACLK	i_axi_clk	i_axi_rstn	AXI4 bus interface — drives AXI slave logic and the AXI-side ports of the async FIFOs (write side of AW-FIFO/WFIFO/ARFIFO; read side of RFIFO/BFIFO)
PCLK	i_apb_clk	i_apb_rstn	APB bus interface — drives the CSR APB access logic and the PCLK side of the 4-phase handshake synchronizers

Domain	Clock signal	Reset signal	Description
SCLK	i_qspi_clk	i_qspi_rstn	QSPI / internal logic clock — drives the QSPI FSM, CSR register file, round-robin arbiter, SCLK-side AXI logic, and the SCLK-side ports of all async FIFOs (read side of AW-FIFO/WFIFO/ARFIFO; write side of RFIFO/BFIFO)

3.2 SCLK and QSPI Output Clock

o_qspi_sclk is a direct wire assignment to i_qspi_clk:

```
assign o_qspi_sclk = i_qspi_clk;
```

There is no clock gating on the SCLK output. The PSRAM device sees a continuous clock whenever i_qspi_clk is toggling. The PSRAM chip-select (o_qspi_csn) gates transactions at the protocol level.

3.3 Reset Strategy

All three resets are active-low and use a synchronous de-assert within their respective clock domain. Each domain has an independent reset:

Reset	Associated clock	Asserted by	De-assert style
i_axi_rstn	i_axi_clk	External (SoC PMU or reset controller)	Synchronous on i_axi_clk rising edge
i_apb_rstn	i_apb_clk	External	Synchronous on i_apb_clk rising edge
i_qspi_rstn	i_qspi_clk	External	Synchronous on i_qspi_clk rising edge

No internal cross-domain reset synchronization is provided. The user is responsible for correct reset sequencing. The recommended order is: 1. Assert all three resets

simultaneously (drive all low). 2. Wait for all clocks to be stable and locked. 3. De-assert `i_qspi_rstn` first (SCLK domain — the register file and FSM must be initialized before the other domains can issue requests). 4. De-assert `i_axi_rstn` and `i_apb_rstn` after `i_qspi_rstn` has been released for at least 2 SCLK cycles.

3.4 Clock Frequency Constraints

The IP imposes no minimum or maximum relationship between ACLK, PCLK, and SCLK frequencies. However, practical performance and latency considerations apply:

- SCLK directly determines PSRAM transaction throughput. Higher SCLK = higher bandwidth. The PSRAM device datasheet specifies the maximum allowable SCLK frequency.
 - ACLK should be $\geq$ SCLK for best AXI throughput. If $ACLK < SCLK$, the async FIFOs may starve the SCLK side.
 - PCLK only affects APB register access latency. The CDC round-trip for each APB access is approximately $2 \times T_{SCLK} + 2 \times T_{PCLK}$. Lower PCLK increases the APB stall duration.
-

3.5 CDC Summary

Crossing	Mechanism	Module
ACLK → SCLK (data)	Async FIFO with toggle-vector occupancy tracking	<code>m_vlsi_async_fifo + m_vlsi_multi_synch</code>
SCLK → ACLK (data)	Async FIFO with toggle-vector occupancy tracking	<code>m_vlsi_async_fifo + m_vlsi_multi_synch</code>
PCLK ↔ SCLK (CSR)	4-phase handshake with 2-stage synchronizers	<code>m_vlsi_qspi_csr + m_vlsi_synch x4</code>

See [04_cdc.md](#) for full CDC detail and required STA constraints.

04 — Clock Domain Crossing (CDC)

The IP crosses between three clock domains. Two independent CDC mechanisms are used: asynchronous FIFOs for ACLK↔SCLK data paths, and a 4-phase handshake for PCLK↔SCLK CSR accesses.

4.1 ACLK ↔ SCLK: Asynchronous FIFO (Toggle-Vector Scheme)

4.1.1 FIFO Instances

Five async FIFOs (`m_vlsi_async_fifo`) bridge ACLK and SCLK:

FIFO	Direction	Data contents
AWFIFO	ACLK → SCLK	{aw_last, aw_id[ID_WD-1:0], aw_addr[ADDR_WD-1:0]}
WFIFO	ACLK → SCLK	{w_last, w_data[DATA_WD-1:0]}
ARFIFO	ACLK → SCLK	{ar_last, ar_id[ID_WD-1:0], ar_addr[ADDR_WD-1:0]}
BFIFO	SCLK → ACLK	{aw_id[ID_WD-1:0], bresp[1:0]}
RFIFO	SCLK → ACLK	{rd_id[ID_WD-1:0], 2'b00, rdata[DATA_WD-1:0], rd_last}

4.1.2 Toggle-Vector Occupancy Scheme

Rather than Gray-coded binary pointers, these FIFOs use a toggle-vector scheme. Each side maintains a shift register of width PARA_DEPTH that tracks occupancy state.

Toggle-Vector Async FIFO CDC Scheme

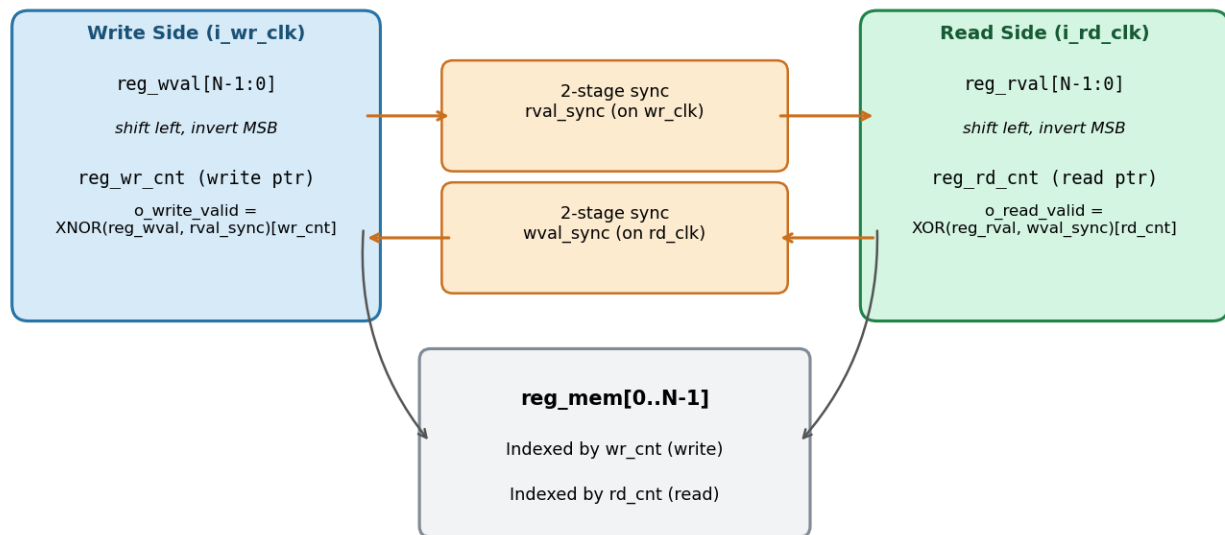


Figure 5: Toggle vector FIFO diagram

Write side logic:

```
// On every write: shift left and invert MSB
reg_wval <= {reg_wval[DEPTH-2:0], ~reg_wval[DEPTH-1]};
```

```
// Slot available when local and synchronized remote vectors match (XNOR)
wr_comb_val = reg_wval ^ rval_sync;
o_write_valid = wr_comb_val[reg_wr_cnt];
```

Read side logic:

```
// On every read: shift left and invert MSB
reg_rval <= {reg_rval[DEPTH-2:0], ~reg_rval[DEPTH-1]};

// Data available when local and synchronized remote vectors differ (XOR)
rd_comb_val = reg_rval ^ wval_sync;
o_read_valid = rd_comb_val[reg_rd_cnt];
```

Each direction uses a `m_vlsi_multi_synch` instance — a `PARAM_DEPTH`-wide 2-stage synchronizer — to safely transfer the occupancy vector to the opposite clock domain.

4.1.3 Required STA Constraints

```
# ACLK → SCLK: vector synchronizer paths
set_max_delay -from [get_clocks ACLK] -to [get_clocks SCLK] \
    [expr {1.5 * $SCLK_period}] -datapath_only

# SCLK → ACLK: vector synchronizer paths
set_max_delay -from [get_clocks SCLK] -to [get_clocks ACLK] \
    [expr {1.5 * $ACLK_period}] -datapath_only
```

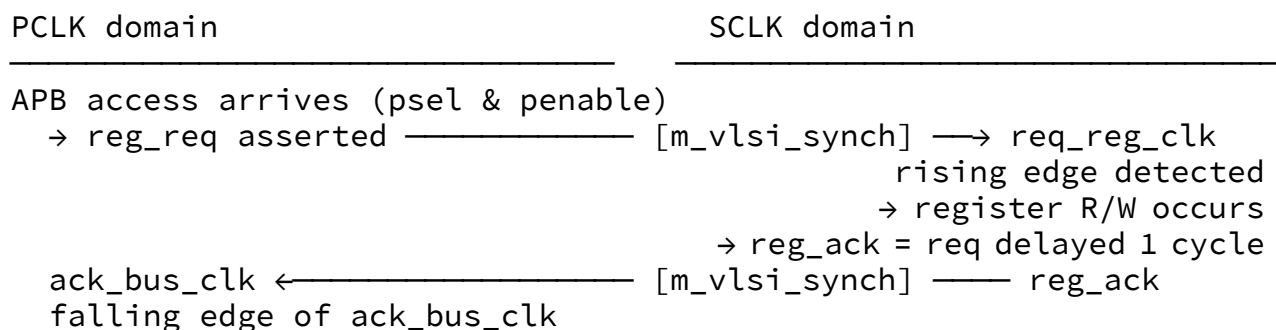
Apply these constraints to all paths through `m_vlsi_multi_synch` instances inside `m_vlsi_async_fifo`.

4.2 PCLK ↔ SCLK: 4-Phase Handshake (CSR)

4.2.1 Mechanism Overview

Every APB register access (read or write) crosses into the SCLK domain via a 4-phase handshake. The APB bus is stalled (`o_pready = 0`) for the entire round-trip CDC latency.

4.2.2 Signal Flow



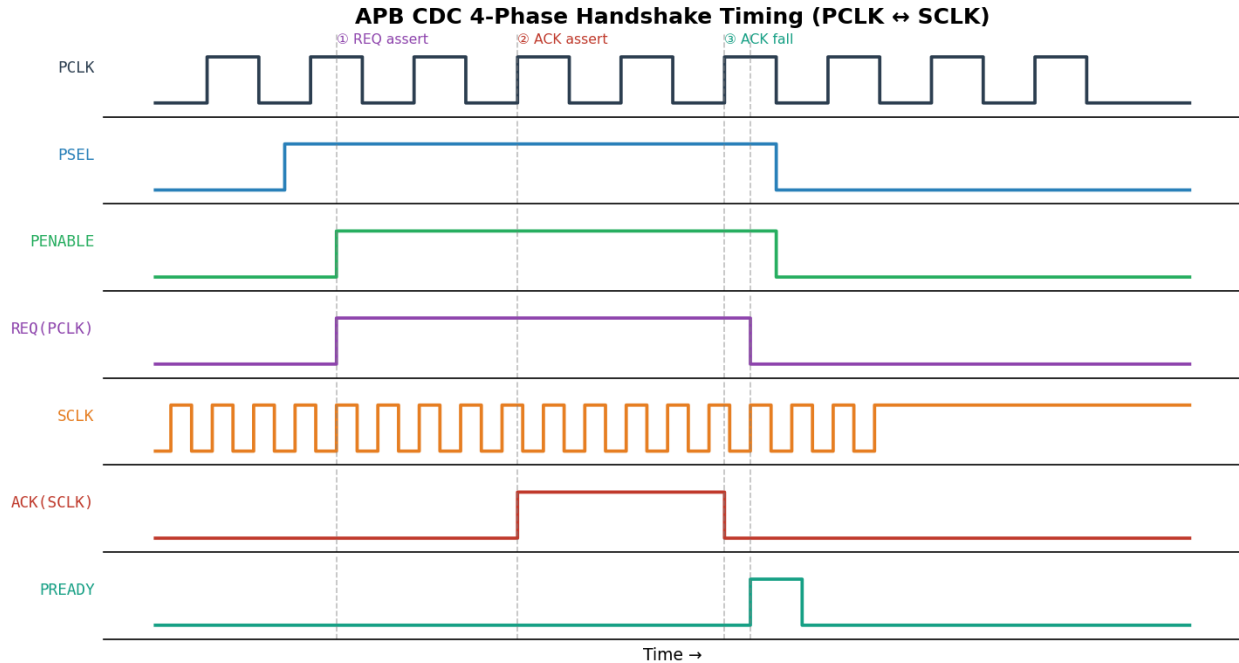


Figure 6: APB CDC timing diagram

- PREADY asserted 1 cycle later
- reg_req cleared on next ack-high cycle

4.2.3 Key Signals in m_vlsi_qspi_csr

Signal	Domain	Description
reg_read_req	PCLK	Read request flop, set on APB read access
reg_write_req	PCLK	Write request flop, set on APB write access
read_req_reg_clk	SCLK	reg_read_req synchronized into SCLK; rising edge triggers read
write_req_reg_clk	SCLK	reg_write_req synchronized into SCLK; rising edge triggers write
read_ack	SCLK	One-cycle delayed copy of read_req_reg_clk; used as ACK back to PCLK
write_ack	SCLK	One-cycle delayed copy of write_req_reg_clk
read_ack_bus_clk	PCLK	read_ack synchronized back to PCLK

write_ack_bus_clk

PCLK

write_ack synchronized
back to PCLK

o_pready asserts one PCLK cycle after the falling edge of read_ack_bus_clk or write_ack_bus_clk.

4.2.4 Latency

Minimum APB stall duration (from penable to pready):

$$T_{\text{stall}} \approx 2 \times T_{\text{SCLK}} + 2 \times T_{\text{PCLK}} + \text{register setup margins}$$

This is the synchronizer pipeline depth (2 stages each direction). At typical frequencies, this is on the order of a few to tens of nanoseconds — negligible for software-driven CSR accesses.

4.2.5 Required STA Constraints

```
# PCLK → SCLK: request synchronizer paths
set_max_delay -from [get_clocks PCLK] -to [get_clocks SCLK] \
    [expr {1.5 * $SCLK_period}] -datapath_only

# SCLK → PCLK: acknowledge synchronizer paths
set_max_delay -from [get_clocks SCLK] -to [get_clocks PCLK] \
    [expr {1.5 * $PCLK_period}] -datapath_only
```

Apply these constraints to all paths through m_vlsi_synch instances inside m_vlsi_qspi_csr.

4.3 CDC Waiver Guidance

If the above constraints are correctly applied and passing in STA:

- All paths through m_vlsi_multi_synch (async FIFO vector synchronizers) can be waived in formal CDC analysis as “synchronized by max-delay constraint.”
- All paths through m_vlsi_synch inside the CSR block can be waived similarly.
- Any remaining CDC paths are either false paths (no data crossing) or are covered by the above mechanisms.

Formal CDC tools (e.g., Synopsys VC Static CDC, Cadence JasperGold CDC) should be configured to recognize these synchronizer cells to avoid false violations.

05 — Configuration & Status Registers (CSR)

All registers are 32 bits wide and accessed through the APB slave interface. The register file is clocked by SCLK; every APB access is synchronized via the 4-phase handshake described in [04_cdc.md](#).

Access type key:

Code	Meaning
RW	Software read/write
RO	Read-only; software writes are ignored
RWI	Read/write by software; hardware may also write (e.g., auto-clear req after ACK)

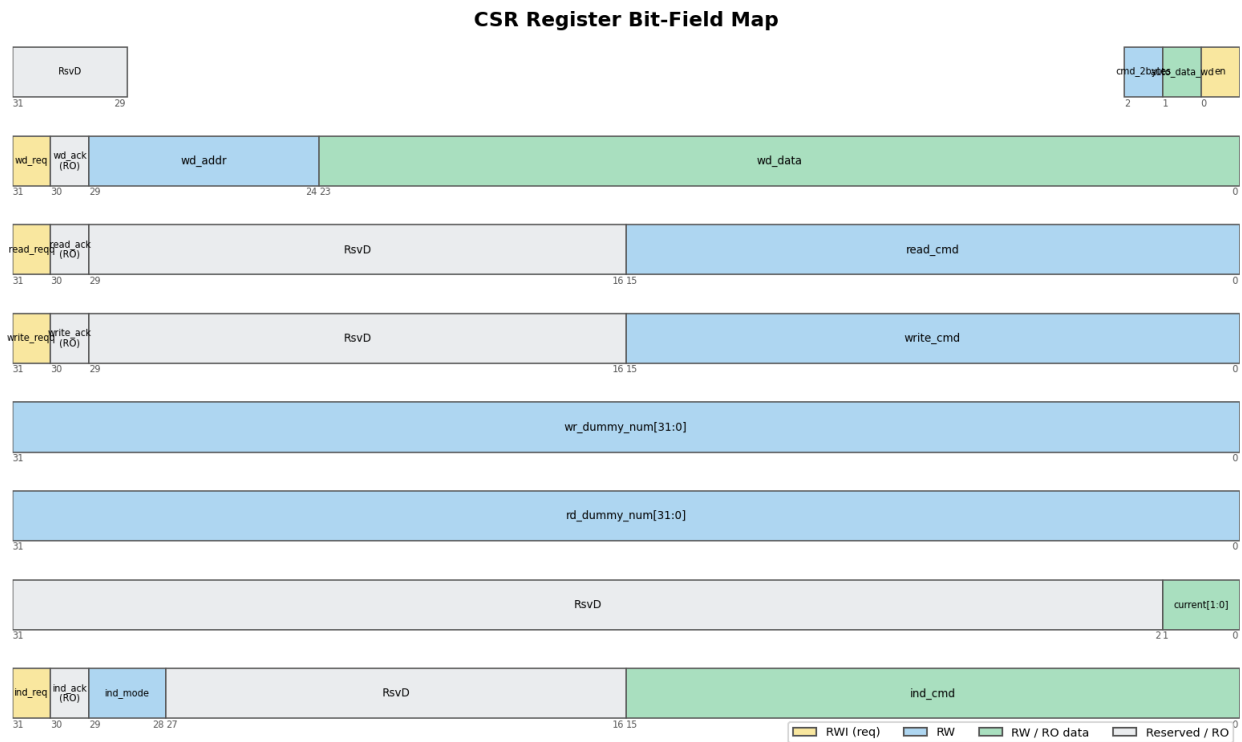


Figure 7: Register bit-fields overview

Register Map

Offset	Name	Description
0x00	ctrl	Global control
0x04	wd	Address/data width descriptor
0x08	read	Read command opcode
0x0C	write	Write command opcode
0x10	wr_dummy	Write dummy cycle count
0x14	rd_dummy	Read dummy cycle count
0x18	mode_status	Current interface mode (read-only)
0x1C	independent	Independent command control

0x00 — ctrl (Control)

Bits	Field	Access	Reset	Description
[31:3]	—	—	0	Reserved. Write as zero; read returns zero.
[2]	cmd_2bytes	RW	0	Command width select. 0: 8-bit (1-byte) command. 1: 16-bit (2-byte) command. Affects how read.cmd, write.cmd, and independent.cmd are interpreted.
[1]	auto_data_wd	RW	0	Auto data-width mode. 1: FSM overrides wd.data with PARA_DATA_WD-1 on every transaction, ignoring the configured wd.data value.
[0]	en	RW	0	IP enable. Must be set to 1 before issuing AXI transactions. When 0, the FSM stays in S_IDLE and ignores all requests.

0x04 — wd (Width Descriptor)

Bits	Field	Access	Reset	Description
[31]	req	RWI	0	Width change request. Write 1 to submit. Hardware clears this bit after the SCLK domain acknowledges the update.
[30]	ack	RO	0	Width change acknowledge. Hardware asserts when the new width values have been latched in the SCLK domain. Cleared automatically when req clears.
[29:24]	addr	RW	0	Address width field (6-bit value, unit: bits). Specifies the number of address bits sent in the ADDR phase of a QSPI transaction.
[23:0]	data	RW	0	Data width field (24-bit value, unit: bits). Specifies the number of data bits per PSRAM access. Ignored when <code>ctrl.auto_data_wd = 1</code> .

Usage: Write `addr` and `data`, then set `req = 1`. Poll `ack` until it asserts. Both `req` and `ack` clear automatically; no explicit clear is needed.

0x08 — read (Read Command)

Bits	Field	Access	Reset	Description
[31]	req	RWI	0	Read command update request. Write 1 to submit. Hardware clears after ACK.
[30]	ack	RO	0	Read command acknowledge. Asserts when the new command is latched in SCLK domain.
[29:16]	—	—	0	Reserved.
[15:0]	cmd	RW	0	Read command opcode. If <code>ctrl.cmd_2bytes</code> = 0, only bits [7:0] are used. If <code>ctrl.cmd_2bytes</code> = 1, all 16 bits are sent MSB-first as two command bytes.

Typical values:

PSRAM mode	Opcode	cmd_2bytes
SPI read	0x03	0
SPI fast read	0x0B	0
QSPI read	0xEB	0
Read ID	Device-specific	0 or 1

0x0C — write (Write Command)

Bits	Field	Access	Reset	Description
[31]	req	RWI	0	Write command update request. Write 1 to submit. Hardware clears after ACK.
[30]	ack	RO	0	Write command acknowledge.

Bits	Field	Access	Reset	Description
[29:16]	—	—	0	Reserved.
[15:0]	cmd	RW	0	Write command opcode. Width governed by <code>ctrl.cmd_2bytes</code> .

Typical values:

PSRAM mode	Opcode	cmd_2bytes
SPI write	0x02	0
QSPI write	0x38	0

0x10 — wr_dummy (Write Dummy Count)

Bits	Field	Access	Reset	Description
[31:0]	num	RW	0	Dummy cycle count for write transactions. The full 32-bit value is stored; the FSM currently consumes only bits [3:0].

FSM scaling:

Mode	Effective dummy cycles
SPI	num[3:0]
QSPI	num[3:0] >> 2 (integer divide by 4)

The 32-bit register width is reserved for future extended dummy cycle configurations. For current implementations, only program bits [3:0] unless otherwise documented.

0x14 — rd_dummy (Read Dummy Count)

Bits	Field	Access	Reset	Description
[31:0]	num	RW	0	Dummy cycle count for read transactions. Same scaling rules as wr_dummy.

FSM scaling: Identical to wr_dummy:

Mode	Effective dummy cycles
SPI	num[3:0]
QSPI	num[3:0] >> 2

0x18 — mode_status (Mode Status)

Bits	Field	Access	Reset	Description
[31:2]	—	—	0	Reserved.
[1:0]	current	RO	0	Current PSRAM interface mode, reflected combinationally from the FSM. 2'b00=SPI, 2'b01=reserved, 2'b10=QSPI, 2'b11=reserved.

This register is read-only and reflects the live mode output from the QSPI FSM (o_csr_mode_status_current). It updates immediately when the FSM transitions mode (e.g., after an independent command completes).

0x1C — independent (Independent Command)

Bits	Field	Access	Reset	Description
[31]	req	RWI	0	Independent command request. Write 1 to issue. Hardware clears after ACK.
[30]	ack	RO	0	Independent command acknowledge. Asserts when FSM has accepted and started the command. Clears automatically.
[29:28]	mode	RW	0	Target interface mode after this command completes. 2'b00=SPI, 2'b10=QSPI.
[27:16]	—	—	0	Reserved.
[15:0]	cmd	RW	0	Command opcode to send (no address or data phase). Width governed by <code>ctrl.cmd_2bytes</code> .

Notes: - An independent command has only a command phase (and optionally a dummy phase based on the current dummy count). No address or data is sent. - If the FSM is processing AXI transactions when `req` is asserted, it will complete all in-flight transactions before servicing the independent command. `ack` will not assert until the FSM is free. - After `ack` asserts, `mode_status.current` will update to reflect the new mode.

Register Access Sequence (req/ack Pattern)

All registers that use the req/ack pattern follow this software sequence:

1. Write the desired field values (`cmd`, `addr`, `data`, `mode`, etc.)
2. Set `req` = 1 in the same or a subsequent APB write
3. Poll the register: read until `ack` = 1

4. Both req and ack clear automatically – no explicit clear needed
5. Proceed with the next operation

Do not issue AXI transactions that depend on the new configuration until ack has been observed.

06 — Functional Description

6.1 QSPI FSM

The QSPI FSM (`m_vlsi_qspi_fsm`) is the core transaction engine, running entirely in the SCLK domain. It sequences all PSRAM commands on the physical QSPI interface.

6.1.1 State Summary

State	Description
S_IDLE	Waiting for a request from the arbiter. CS is deasserted.
S_SET_UP	Captures CSR snapshot (command opcode, data/address width, dummy count). Acknowledges pending CSR req bits.
S_WAIT_TRANS	Asserts <code>o_ax_write_valid</code> to accept write data from SCLK-side AXI logic. Waits until write data is available.
S_IND_CMD	Sends an independent (command-only) transaction. No address or data phase.
S_CMD	Drives the command phase on QSPI pads. Shifts out 8 or 16 bits depending on <code>ctrl.cmd_2bytes</code> .
S_ADDR	Drives the address phase on QSPI pads. Number of bits set by <code>wd.addr</code> .
S_DUMMY	Drives the dummy phase. Data pins tri-stated. Cycle count derived from <code>wr_dummy</code> / <code>rd_dummy</code> scaled by mode.
S_WRITE	Drives write data onto QSPI pads. Number of bits set by <code>wd.data</code> (or <code>PARA_DATA_WD-1</code> if <code>auto_data_wd</code>).
S_READ	Samples read data from QSPI pads into shift register. Returns data to SCLK-side AXI logic.

6.1.2 State Transitions

S_IDLE

- └ [independent req & no pending AXI] → S_SET_UP → S_IND_CMD → S_IDLE
- └ [write granted by arbiter] → S_SET_UP → S_WAIT_TRANS → S_CMD → S_ADDR → S_WRITE → S_READ → S_IDLE
- └ [read granted by arbiter] → S_SET_UP → S_CMD → S_ADDR → S_DUMMY → S_READ → S_IDLE

Idle
SW can configures all CSR register, and configure to enable the csr_ctrl_en at the last.

Set_up
Check the all "req" bits, if the csr_*_req and there is no remain items (FIFO read and write are empty), IP will set the csr*_ack = 1

Wait_trans
If csr_independent_req and ack == 1, IP will move to "Ind_cmd" state, unless IP wait transaction for read and write

Ind_cmd
CS is change 1 - 0, reg_cnt_cmd counts down, command is sent to IOPAD, when the reg_cnt_cmd == 0, IP returns to "set_up" state

Write/Read
set the corresponding register
reg_in_write/reg_in_read, which are used to mode data from FIFO to IOPAD in write phase or shift data from IOPAD to internal register in read phase

Cmd
CS changes 1-0 and reg_cnt_cmd is set to csr value before this state. Cmd is sent out to IOPAD until the reg_cnt_cmd = 0

Addr
reg_cnt_addr is set to csr value before this state. Addr is sent out to IOPAD until the reg_cnt_addr = 0

Dummy
reg_cnt_dummy is set to csr value before this state. the value of IOPAD will be dummy until the reg_cnt_dummy = 0

Data
reg_cnt_data is set to csr value before this state. the data will shift out to IOPAD until the reg_cnt_data = 0

FSM

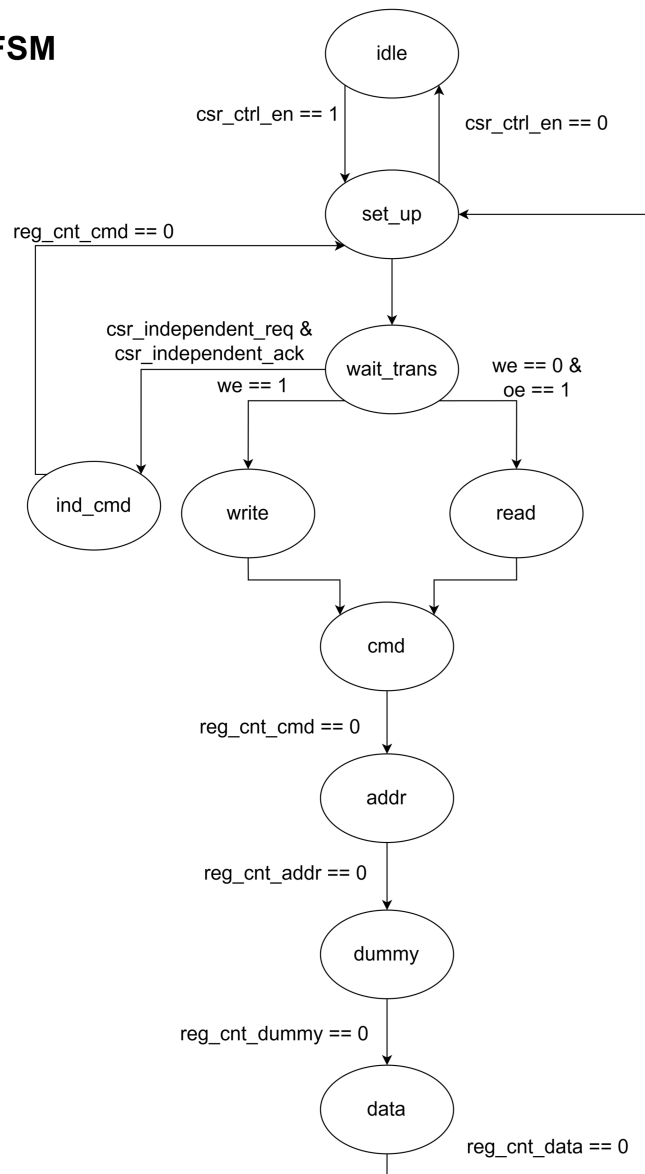


Figure 8: FSM state diagram

- CS (o_qspi_csn) asserts at the start of S_CMD or S_IND_CMD and deasserts at the end of the final data/command phase.
- If ctrl.en = 0, the FSM stays in S_IDLE and ignores all requests.

6.1.3 Phase Details

Command Phase (S_CMD, S_IND_CMD): - SPI mode: DQ[0] driven, DQ[1:3] tri-stated - QSPI mode: DQ[3:0] all driven - 8 bits sent if ctrl.cmd_2bytes = 0; 16 bits (2 bytes, MSB first) if ctrl.cmd_2bytes = 1

Address Phase (S_ADDR): - SPI mode: DQ[0] driven serially - QSPI mode: DQ[3:0] driven (4 bits per clock) - Number of bits: wd.addr

Dummy Phase (S_DUMMY): - All DQ pins tri-stated - Cycle count: rd_dummy.num[3:0] (read) or wr_dummy.num[3:0] (write), scaled by mode: - SPI: value used directly - QSPI: value divided by 4

Data Phase (S_WRITE / S_READ): - SPI write: DQ[0] driven serially - QSPI write: DQ[3:0] driven (4 bits per clock) - SPI read: DQ[1] sampled serially - QSPI read: DQ[3:0] sampled (4 bits per clock) - Number of bits: wd.data (or PARA_DATA_WD-1 when auto_data_wd = 1)

6.2 Arbitration

The arbiter (m_vlsi_arbiter) performs round-robin selection between concurrent read and write requests in the SCLK domain.

6.2.1 Arbitration Logic

```
o_arb_sel      = i_req_read  & (~i_req_write | arb_toggle); // 1 = read selected
o_arb_write_en = i_req_write & (~i_req_read  | ~arb_toggle); // 1 = write selected
```

arb_toggle flips after each granted transaction, ensuring neither read nor write can starve.

6.2.2 Request Conditions

```
req_write = ~awfifo_empty & ~wfifo_empty & (~aw_last | ~bfifo_full)
req_read  = ~arfifo_empty & ~rd_pending  & ~rfifo_full
```

- A write request requires both an address (AWFIFO) and data (WFIFO) beat to be available, and space in BFIFO for the response.
- A read request requires an address (ARFIFO) beat, no read already in flight (rd_pending), and space in RFIFO for the response.

AXI → QSPI Data-Flow Overview

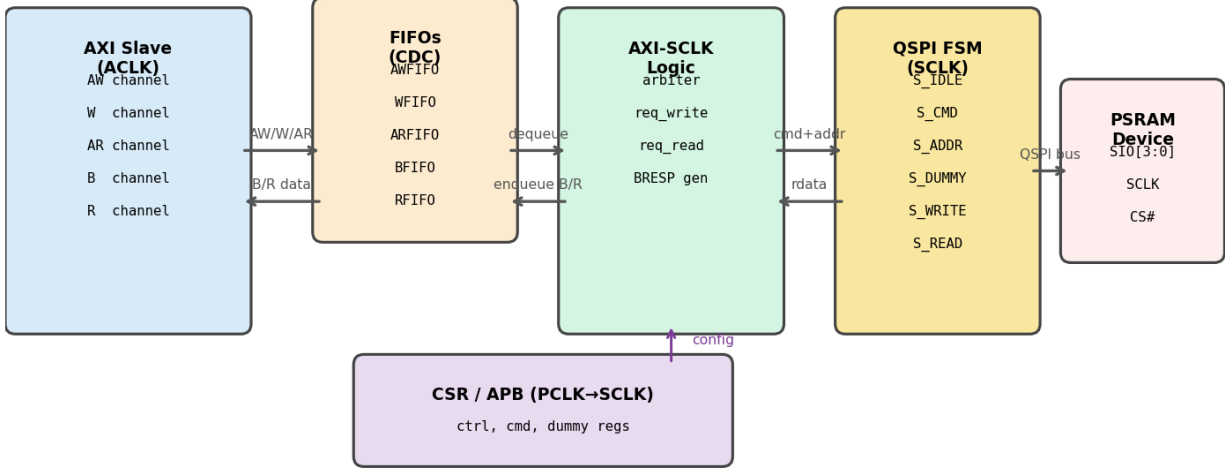


Figure 9: AXI write dataflow

6.3 Write Flow

1. AXI address channel: `m_vlsi_axfsm` (AW instance) accepts the AXI burst descriptor, generates per-beat addresses for the burst, and pushes each `{aw_last, aw_id, aw_addr}` entry into AWFIFO.
2. AXI data channel: `m_vlsi_axi_bus_logic` accepts W-channel beats and pushes `{w_last, w_data}` into WFIFO.
3. Arbitration: The arbiter grants a write when `req_write` is asserted. `m_vlsi_axi_sclk_logic` pops AWFIFO and WFIFO and presents the data to the FSM.
4. FSM sequencing: `S_SET_UP` → `S_WAIT_TRANS` → `S_CMD` → `S_ADDR` → `S_DUMMY` → `S_WRITE`
5. Response: After the last write beat, `{aw_id, bresp}` is pushed into BFIFO. `m_vlsi_axi_bus_logic` pops BFIFO and drives the AXI B channel.

Write response codes:

o_bresp	Condition
2'b00 (OKAY)	AW burst length matches W beat count
2'b10 (SLVERR)	aw_last and w_last mismatch — malformed burst

6.4 Read Flow

1. AXI address channel: `m_vlsi_ax fsm` (AR instance) generates per-beat addresses and pushes `{ar_last, ar_id, ar_addr}` into ARFIFO.
2. Arbitration: The arbiter grants a read when `req_read` is asserted. `m_vlsi_axi_sclk_logic` pops ARFIFO and presents the address to the FSM.
3. FSM sequencing: `S_SET_UP` $\rightarrow$ `S_CMD` $\rightarrow$ `S_ADDR` $\rightarrow$ `S_DUMMY` $\rightarrow$ `S_READ`

4. Data return: Read data is pushed into RFIFO as {rd_id, 2'b00, rdata, rd_last}.
5. AXI data channel: m_vlsi_axi_bus_logic pops RFIFO and drives the AXI R channel.

6.5 Independent Command Flow

An independent command is a command-only transaction — no address or data phase. It is primarily used to switch the PSRAM from SPI to QSPI mode (or vice versa).

Sequence: 1. Software writes `independent.cmd` and `independent.mode`, then sets `independent.req = 1`. 2. If the FSM has in-flight AXI transactions, it completes them first. 3. FSM enters `S_IND_CMD`, drives the command on QSPI pads, then updates the internal interface mode register to `independent.mode`. 4. FSM asserts `independent.ack`. Both `req` and `ack` clear automatically. 5. `mode_status.current` updates to reflect the new mode.

After a SPI→QSPI mode switch, the user should update `read.cmd` and `write.cmd` to the QSPI equivalents before issuing further AXI transactions.

6.6 XIP Mode

XIP (Execute-In-Place) behavior is inferred from the AXI burst length — no dedicated CSR bit is required.

AXI transaction type	FSM behavior
Single-beat (<code>awlen/arlen = 0</code>)	Standard single-word read or write
Multi-beat burst (<code>awlen/arlen > 0</code>)	Streaming (XIP) read or write — address auto-increments per beat

The user must ensure the PSRAM device is in the correct operating mode (e.g., QSPI) before issuing burst transactions intended for XIP.

6.7 auto_data_wd Mode

When `ctrl.auto_data_wd = 1`, the FSM ignores the `wd.data` register and instead forces the data width to `PARAM_DATA_WD - 1` on every `S_WRITE` or `S_READ` entry. This is useful when the PSRAM always transfers full-width words and per-transaction width control is not needed.

6.8 Write Response Error (SLVERR)

`BRESP = SLVERR (2'b10)` is generated when:

$(aw_last \text{ from } AWFIFO) \text{ XOR } (w_last \text{ from } WFIFO) \neq 0$

This condition detects a mismatch between the declared AW burst length and the actual number of W-channel beats received. It protects the system against malformed bursts that could leave the FSM in an inconsistent state.

07 — Software Integration Guide

7.1 Initialization Sequence

Perform this sequence after all clocks are stable and before issuing any AXI transactions.

Step 1: Assert all resets (hold `i_qspi_rstn`, `i_axi_rstn`, `i_apb_rstn` low)
Step 2: Start all clocks and wait for stability (PLL lock, etc.)
Step 3: De-assert `i_qspi_rstn` first; wait ≥ 2 SCLK cycles
Step 4: De-assert `i_axi_rstn` and `i_apb_rstn`
Step 5: Configure registers via APB (see below)
Step 6: Set `ctrl.en = 1`

7.1.1 Register Configuration Order

- a. Write `wr_dummy.num` – dummy cycles for writes
 - b. Write `rd_dummy.num` – dummy cycles for reads
 - c. Write `write.cmd`, then set `write.req = 1`
Poll `write.ack` until asserted
 - d. Write `read.cmd`, then set `read.req = 1`
Poll `read.ack` until asserted
 - e. Write `wd.addr` and `wd.data`, then set `wd.req = 1`
Poll `wd.ack` until asserted
 - f. If the PSRAM starts in SPI mode and QSPI is desired:
Write `independent.cmd = <PSRAM QPI-enable opcode>`
Write `independent.mode = 2'b10` (QSPI)
Set `independent.req = 1`
Poll `independent.ack` until asserted
 - g. Write `ctrl.en = 1`

 `wr_dummy` and `rd_dummy` do not use the req/ack mechanism — they are direct register writes that take effect on the next transaction.
-

7.2 Normal AXI Read/Write

After initialization, the CPU issues standard AXI4 transactions. No additional software intervention is required per transaction.

- AXI address maps directly to the PSRAM physical address — no offset or shift applied.
 - Burst types FIXED, INCR, and WRAP are all supported.
 - Monitor `o_bresp` on the B channel; `SLVERR` (`2'b10`) indicates a burst-length mismatch (see [06_functional.md §6.8](#)).
-

7.3 Changing Command Opcodes at Run Time

To switch to a different command opcode (e.g., temporarily using Read-ID):

1. Write the new opcode to `read.cmd` (or `write.cmd`)
2. Set `read.req = 1` (or `write.req = 1`)
3. Poll `read.ack` until asserted
4. Issue the special AXI transaction
5. Write the standard opcode back; repeat steps 1–3 to restore

If the address or data width also needs to change for this command:

1. Write `wd.addr` and/or `wd.data` to the new values
2. Set `wd.req = 1`
3. Poll `wd.ack` until asserted
4. Issue the transaction
5. Restore `wd.addr/wd.data` and repeat the req/ack cycle

Do not issue AXI transactions between setting `req` and observing `ack`. The new configuration is not guaranteed to be active in the SCLK domain until `ack` asserts.

7.4 Independent (Command-Only) Transaction

Used for PSRAM control commands that have no data payload — e.g., mode switching, reset, burst-length configuration.

1. Write `independent.cmd = <opcode>`
 2. Write `independent.mode = target mode after this command`
(`2'b00 = SPI`, `2'b10 = QSPI`)
 3. Set `independent.req = 1`
 4. Poll `independent.ack` until asserted
(Any pending AXI transactions complete first; `ack` may be delayed)
 5. `req` and `ack` clear automatically — no explicit write needed
 6. Read `mode_status.current` to confirm the new mode
-

7.5 Switching from SPI to QSPI Mode

Full sequence for transitioning a PSRAM device from SPI to QSPI mode:

1. Confirm the PSRAM is currently in SPI mode
(default after power-on or reset for most devices)

2. Issue the mode-switch independent command:
independent.cmd = <device QPI-enter opcode, e.g. 0x35>
independent.mode = 2'b10
independent.req = 1
→ poll independent.ack
 3. Update read and write command opcodes:
read.cmd = <QSPI read opcode, e.g. 0xEB>
read.req = 1 → poll read.ack
write.cmd = <QSPI write opcode, e.g. 0x38>
write.req = 1 → poll write.ack
 4. Update dummy cycle counts if needed for the new mode:
rd_dummy.num = <new value>
wr_dummy.num = <new value>
 5. Resume normal AXI transactions (now in QSPI mode)
-

7.6 Switching from QSPI to SPI Mode

1. Issue the mode-switch independent command:
independent.cmd = <device SPI-exit opcode, e.g. 0xF5>
independent.mode = 2'b00
independent.req = 1
→ poll independent.ack
 2. Update read and write command opcodes to SPI values:
read.cmd = <SPI read opcode, e.g. 0x03>
read.req = 1 → poll read.ack
write.cmd = <SPI write opcode, e.g. 0x02>
write.req = 1 → poll write.ack
 3. Update dummy cycle counts if needed
 4. Resume normal AXI transactions (now in SPI mode)
-

7.7 APB Access Latency Notes

Every APB register access (read or write) stalls on PREADY for the duration of the 4-phase CDC handshake.

Minimum stall $\approx 2 \times T_{\text{SCLK}} + 2 \times T_{\text{PCLK}}$

Example: SCLK = 100 MHz (T = 10 ns), PCLK = 50 MHz (T = 20 ns):

Minimum stall $\approx 2 \times 10 + 2 \times 20 = 60 \text{ ns} \approx 3 \text{ PCLK cycles}$

Software must not assume zero-latency APB access. The APB infrastructure handles the stall transparently if PREADY is wired to the bus controller. In bare-metal drivers, ensure the APB read function waits for PREADY before capturing PRDATA.

7.8 Polling vs. Interrupt-Driven Operation

The IP does not generate interrupts. All req/ack synchronization is polling-based:

- For initialization sequences, simple polling loops are sufficient since the latency is short (tens of nanoseconds per access).
 - For high-frequency runtime command changes, consider using a DMA or hardware sequencer to avoid polling overhead in latency-sensitive paths.
-

7.9 Bare-Metal Driver Pseudocode

```
// Write a CSR register with req/ack
void csr_write_with_ack(uint32_t offset, uint32_t value) {
    apb_write(BASE + offset, value | (1u << 31)); // set req
    while (!(apb_read(BASE + offset) & (1u << 30))) // poll ack
        ;
}

// Initialize the IP for QSPI operation
void qspi_psram_init(void) {
    apb_write(BASE + 0x10, 0); // wr_dummy = 0
    apb_write(BASE + 0x14, 6); // rd_dummy = 6 (QSPI: 6>>2 = 1 cycle)

    csr_write_with_ack(0x0C, 0x38); // write.cmd = 0x38 (QSPI write)
    csr_write_with_ack(0x08, 0xEB); // read.cmd = 0xEB (QSPI read)
    csr_write_with_ack(0x04, (24u << 24) | 32u); // wd: addr=24 bits, data=32 bits

    // Issue SPI→QSPI mode switch command (e.g., 0x35 for some PSRAMs)
    csr_write_with_ack(0x1C, (0x35u) | (0x2u << 28)); // cmd=0x35, mode=QSPI

    apb_write(BASE + 0x00, 0x1); // ctrl.en = 1
}
```

08 — STA Constraints Reference

This section lists all timing constraints required for correct static timing analysis of the IP. Apply these constraints in your synthesis and STA scripts.

8.1 ACLK ↔ SCLK (Async FIFO Toggle-Vector Synchronizers)

These constraints apply to the `m_vlsi_multi_synch` instances inside each `m_vlsi_async_fifo`. They cover the occupancy vector paths that cross between ACLK and SCLK.

```
# ACLK → SCLK: write-side vector to read side
set_max_delay \
    -from [get_clocks {i_axi_clk}] \
    -to   [get_clocks {i_qspi_clk}] \
    [expr {1.5 * $SCLK_period}] \
    -datapath_only

# SCLK → ACLK: read-side vector to write side
set_max_delay \
    -from [get_clocks {i_qspi_clk}] \
    -to   [get_clocks {i_axi_clk}] \
    [expr {1.5 * $ACLK_period}] \
    -datapath_only
```

The delay value is 1.0 to 1.5 times the receiving clock period. Using 1.5× provides margin for synchronizer setup time.

8.2 PCLK ↔ SCLK (CSR 4-Phase Handshake Synchronizers)

These constraints apply to the `m_vlsi_synch` instances inside `m_vlsi_qspi_csr`. They cover the req/ack handshake signals crossing between PCLK and SCLK.

```
# PCLK → SCLK: request signals from APB domain to register domain
set_max_delay \
    -from [get_clocks {i_apb_clk}] \
    -to   [get_clocks {i_qspi_clk}] \
    [expr {1.5 * $SCLK_period}] \
    -datapath_only

# SCLK → PCLK: acknowledge signals from register domain back to APB domain
set_max_delay \
    -from [get_clocks {i_qspi_clk}] \
    -to   [get_clocks {i_apb_clk}] \
    [expr {1.5 * $PCLK_period}] \
    -datapath_only
```

8.3 False Path Declarations

The following paths are structurally asynchronous and should be declared as false paths to prevent STA from reporting spurious violations:

```
# No timing relationship between AXI clock and APB clock
set_false_path \
    -from [get_clocks {i_axi_clk}] \
    -to   [get_clocks {i_apb_clk}]

set_false_path \
    -from [get_clocks {i_apb_clk}] \
    -to   [get_clocks {i_axi_clk}]
```

ACLK and PCLK never directly exchange data — all AXI↔APB paths are either through independent logic or through the SCLK domain as the intermediate. These false paths prevent STA tools from trying to analyze a nonexistent relationship.

8.4 Reset Path Constraints

All resets are synchronous within their domain. No special `set_false_path` for reset paths is required unless the tool flags asynchronous reset removal paths across domains.

If the tool reports a CDC violation on reset paths:

```
# Reset paths are synchronous within each domain – no cross-domain reset path
set_false_path -from [get_ports {i_axi_rstn}] -to [get_clocks {i_qspi_clk}]
set_false_path -from [get_ports {i_axi_rstn}] -to [get_clocks {i_apb_clk}]
set_false_path -from [get_ports {i_apb_rstn}] -to [get_clocks {i_axi_clk}]
set_false_path -from [get_ports {i_apb_rstn}] -to [get_clocks {i_qspi_clk}]
set_false_path -from [get_ports {i_qspi_rstn}] -to [get_clocks {i_axi_clk}]
set_false_path -from [get_ports {i_qspi_rstn}] -to [get_clocks {i_apb_clk}]
```

8.5 Synchronizer Cell Attributes

To ensure synthesis does not optimize away synchronizer flip-flop stages, apply the `DONT_TOUCH` or equivalent attribute to all `m_vlsi_synch` and `m_vlsi_multi_synch` instances:

```
# Synopsys DC / Fusion Compiler
set_dont_touch [get_cells -hierarchical -filter {REF_NAME == m_vlsi_synch}]
set_dont_touch [get_cells -hierarchical -filter {REF_NAME == m_vlsi_multi_synch}]
```

Or in RTL (if synthesis attributes are used):

```
(* dont_touch = "true" *) m_vlsi_synch u_synch ( ... );
```

8.6 Constraint Summary Table

Path	Constraint type	Value
ACLK → SCLK (FIFO vectors)	set_max_delay -datapath_only	1.5 × T_SCLK
SCLK → ACLK (FIFO vectors)	set_max_delay -datapath_only	1.5 × T_ACLK
PCLK → SCLK (CSR req)	set_max_delay -datapath_only	1.5 × T_SCLK
SCLK → PCLK (CSR ack)	set_max_delay -datapath_only	1.5 × T_PCLK
ACLK ↔ PCLK (no data path)	set_false_path	—
Cross-domain reset paths	set_false_path	—

8.7 CDC Tool Waiver Guidance

After applying the above constraints, all remaining CDC violations reported by formal CDC tools (Synopsys VC Static, Cadence JasperGold CDC) can be waived as follows:

Violation type	Waiver justification
FIFO pointer / occupancy vector	Constrained by set_max_delay -datapath_only; 2-stage synchronizer in m_vlsi_multi_synch
CSR req/ack signals	Constrained by set_max_delay -datapath_only; 2-stage synchronizer in m_vlsi_synch
ACLK ↔ PCLK paths	No direct data crossing; declared set_false_path

Configure your CDC tool to recognize m_vlsi_synch and m_vlsi_multi_synch as known synchronizer primitives to suppress false-positive reconvergence or multi-bit CDC violations.